General AI Fluency — Capstone Submission

Ziyad Alboghdady · Backend AI Engineering, July 2026 cohort Live portfolio: https://ziyad-alboghdady.netlify.app

---

1. How to add the next case study — the concrete note

The site is nine pages of static HTML generated by build_site.py (~300 lines, Python standard library only). Every case study on the site is **one dictionary entry in the CASES list**. The projects index, the two featured cards on the homepage, the sitemap and the next/previous links between cases all read from that same list, so adding one entry updates all of them.

The steps, exactly:

1. Open build_site.py. Copy the last entry in CASES.
2. Fill the three beats — the shape is fixed, and that is the point:
   - **The problem** — what was actually hard. Not "I wanted to learn X."
   - **What I did & decided** — bullets, each one a *decision with its trade-off*, not a list of technologies.
   - **What came of it** — what changed as a result. If nothing did, the case is not ready.
3. Set stack, role, period, repo, and a one-sentence summary. Write the summary last, once the case is clear — it is what appears on the cards.
4. Set artifact to the real capture the case needs. **Take that capture before writing the prose.** If I cannot capture it, the case is not finished.
5. If it is now my strongest work, move it to position 1 in CASES. The homepage leads with the first two automatically.
6. python3 build_site.py → preview with python3 -m http.server from inside portfolio-site/ → drag the folder to Netlify.

About twenty minutes of mechanics. The writing is the work; the build is not. That ratio is deliberate — I built it this way so the cost of a new case study is the thinking, never the plumbing.

2. The next real piece of work

Integration tests for the Task Management API, and a fifth case study about them.

I picked this because it closes the one gap I cannot defend. My homepage claims I build *reliable, **testable*** backends. Four case studies describe layering, boundaries and schema design — and not one of them shows a test. A reviewer who asks "where are the tests?" is asking a fair question, and right now my answer is a sentence rather than a link.

The shape of the work:

- WebApplicationFactory against the real PostgreSQL 16 container that already runs in docker compose, so the tests exercise the actual stack rather than a mock of it.
- The cases worth testing are the promises the case study already makes:
  - a request for another user's task returns **404, not 403**
  - a missing Authorization header returns **401**
  - Supabase unreachable returns **503, not 401**
- Those three are the architectural claims. If they are true, a test should prove it. If a test turns out to be awkward to write, that is information about the design, and it goes in the case study.

Why not the EF Core migration (which was my earlier plan): it is already done. The Task API runs on .NET 9, EF Core and PostgreSQL 16 in Docker with Supabase auth. The *reversibility* claim has been paid off. What has not been paid off is *testable*.

Target: within 6 weeks of the internship ending.

3. The reminder — a real one, not an intention

- **Recurring calendar event: first Saturday of every month, 10:00 — "Portfolio: one case study step."** Recurring on purpose, so a missed month does not end the habit.
- The first three occurrences carry the *specific* next action rather than the vague goal:
  1. Stand up WebApplicationFactory against the compose database
  2. Write the 404-not-403 test
  3. Write the case study
- A NEXT.md at the root of the portfolio repo holds the single next action, so opening the folder answers "what was I doing" in one line.

Evidence: screenshot of the recurring calendar event — evidence/reminder.png.

A vague intention to "keep it updated" is how portfolios die. A dated, repeating event with a named action is the difference, and the naming matters more than the dating: "work on portfolio" is something I will snooze, "write the 404-not-403 test" is something I will either do or consciously skip.

4. The build context is preserved

The Claude Project **"Portfolio — Ziyad"** stays, holding as project knowledge:

- the identity kit (fonts, hex codes, logo spec, standing style note)
- the four case studies in three-beat form
- the content map and the one-line claim
- build_site.py and style.css

The standing style note is set as the project's instructions, so every future request inherits the visual system instead of me restating it. **That context is the actual asset.** The next case study is a conversation — *"here's what I built, here's the capture, draft the three beats in the*

existing voice" — not a rebuild. Recreating that context from scratch would cost more than writing the case study itself.

It is also backed up outside Claude: AGENT_INSTRUCTIONS.md, the identity kit and the content map are committed as files in the repo, so if the Project is ever lost it can be rebuilt in five minutes from the folder rather than from memory.

Against the pass / revise criteria

Criterion	Where it is met
A concrete "how to add the next case" note, not a vague intention	§1 — six numbered steps against a named file and a named list
A specific next piece of work named, with a real reminder set	§2 names the work and the three tests; §3 is a recurring calendar event with per-occurrence actions
Build context preserved so future updates are cheap	§4 — Claude Project retained and mirrored as files in the repo